1. Create NumPy arrays from Python Data Structures, Intrinsic NumPy objects and Random Functions.

```
import numpy as np

python_list = [1, 2, 3, 4, 5]

array_from_list = np.array(python_list)

print("Array from Python list:\n",array_from_list)


array_from_arange = np.arange(0, 10, 2)

print("\nArray using np.arange:\n",array_from_arange)


array_from_linspace = np.linspace(0, 1, 5)

print("\nArray using np.linspace:\n",array_from_linspace)


random_array = np.random.rand(3, 3)

print("\nRandom array (3x3) using np.random.rand:\n",random_array)


random_int_array = np.random.randint(1, 11, size=(2, 4))

print("\nRandom integer array (2x4) using
np.random.randint:\n",random_int_array)
```

output:

```
Array from Python list:
 [1 2 3 4 5]

Array using np.arange:
 [0 2 4 6 8]

Array using np.linspace:
 [0.   0.25 0.5  0.75 1.  ]

Random array (3x3) using np.random.rand:
 [[0.92357495 0.33721704 0.96490083]
 [0.85332567 0.03621806 0.6147754 ]
 [0.62227778 0.64308072 0.2289968 ]]

Random integer array (2x4) using np.random.randint:
 [[ 2 10 10  8]
 [ 8  1  1  9]]
```

2. Manipulation of NumPy arrays- Indexing, Slicing, Reshaping, Joining and Splitting.

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60])
print("Array element",arr)
print("\nIndexing: 3rd element ", arr[2])  # 30
print("Indexing: last element",arr[5])

print("\nSlicing:", arr[1:4])  # [20 30 40]
print("Slicing with negitive index",arr[-3:-1])

reshaped = arr.reshape(2, 3)
print("\nReshaped (2x3):\n", reshaped)

a = np.array([1, 2])
b = np.array([3, 4])
joined = np.concatenate((a, b))
print("\nJoined:", joined)

split = np.array_split(arr, 3)
print("\nSplitting:")
for part in split:
    print(part)
```

output:

```
Array element [10 20 30 40 50 60]

Indexing: 3rd element  30
Indexing: last element 60

Slicing: [20 30 40]
```

```
Slicing with negitive index [40 50]

Reshaped (2x3):
 [[10 20 30]
 [40 50 60]]

Joined: [1 2 3 4]

Splitting:
[10 20]
[30 40]
[50 60]
```

3. Computation on NumPy arrays using Universal Functions and Mathematical methods.

```
import numpy as np

arr1 = np.array([6, 3, 9, 16, 20])

arr2 = np.array([1, 2, 7, 3, 15])


print("Elements of Array 1:",arr1)

print("Elements of Array 2:",arr2)


print("Sum of arr1 and arr2:", np.add(arr1,arr2))

print("Difference of arr1 and arr2:", np.subtract(arr1,arr2))

print("Product of arr1 and arr2:",np.multiply(arr1,arr2))

print("Square roots:",np.sqrt(arr1))


print("\nSum:", arr1.sum())

print("Mean:", arr1.mean())

print("Max:", arr1.max())

print("Powers:",np.power(arr1, 2))

print("Standard Deviation:", arr1.std())
```

output:

```
Elements of Array 1: [ 6  3  9 16 20]
Elements of Array 2: [ 1  2  7  3 15]
Sum of arr1 and arr2: [ 7  5 16 19 35]
```

```
Difference of arr1 and arr2: [ 5  1  2 13  5]
Product of arr1 and arr2: [  6   6  63  48 300]
Square roots: [2.44948974 1.73205081 3.         4.         4.47213595]

Sum: 54
Mean: 10.8
Max: 20
Powers: [ 36   9  81 256 400]
Standard Deviation: 6.305553108173779
```

4. Create Pandas Series and Data Frame from various inputs.

```
import pandas as pd

import numpy as np

series_from_list = pd.Series([10, 20, 30, 40, 50])

print("Series from List:")

print(series_from_list)

series_from_dict = pd.Series({"A": 100, "B": 200, "C": 300})

print("\nSeries from Dictionary:")

print(series_from_dict)


data = {

   'Name': ['Alice', 'Bob', 'Charlie'],

   'Age': [25, 30, 35],

   'City': ['New York', 'Los Angeles', 'Chicago']

}

df_from_dict = pd.DataFrame(data)

print("\nDataFrame from Dictionary:")

print(df_from_dict)


data_array = np.array([["Alice", 25, "New York"], ["Bob", 30, "Los Angeles"],
["Charlie", 35, "Chicago"]])

df_from_array = pd.DataFrame(data_array, columns=["Name", "Age", "City"])
```

```
print("\nDataFrame from NumPy Array:")
print(df_from_array)
```

output:

```
Series from List:
0    10
1    20
2    30
3    40
4    50
dtype: int64

Series from Dictionary:
A    100
B    200
C    300
dtype: int64

DataFrame from Dictionary:
      Name  Age         City
0    Alice   25     New York
1      Bob   30  Los Angeles
2  Charlie   35      Chicago

DataFrame from NumPy Array:
      Name Age         City
0    Alice  25     New York
1      Bob  30  Los Angeles
2  Charlie  35      Chicago
```

5. Import any CSV file to Pandas Data Frame and perform the following:
   a. Visualize the first and last 5 records
   b. Get the shape, index and column details
   c. Select/Delete the records (rows)/columns based on conditions.
   d. Perform ranking and sorting operations.
   e. Do require statistical operations on the given column

csv file:

Name, Age, City
Akash, 25, Kurnool
Balu, 30, Ananthapur
Charlie, 35, Nandyal
Dally ,40, Hyderabad
Evangilin,45, Chennai
Florence,36, Bangalore

Python code:

```
import pandas as pd
df = pd.read_csv('d:\myfile.csv')
```

a.
```
print("First 5 records:")
print(df.head(5))

print("\nLast 5 records:")
print(df.tail(5))
```

b.
```
print("\nShape of the DataFrame:", df.shape)  # Returns (rows, columns)
print("\nIndex of the DataFrame:", df.index)  # Returns index
print("\nColumns of the DataFrame:", df.columns)  # Column names
```

c.
```
selected_rows = df[df['Age'] > 30]
print("\nRows where 'Age' > 30:")
print(selected_rows)

df_dropped_column = df.drop('City', axis=1)  # Drops the 'City' column
print("\nDataFrame after dropping 'City' column:")
print(df_dropped_column)
```

d.
```
sorted_df = df.sort_values(by='Age', ascending=True)
print("\nDataFrame sorted by 'Age' (ascending):")
print(sorted_df)

df['Age Rank'] = df['Age'].rank()
print("\nDataFrame with 'Age Rank' column:")
print(df)
```

e.
```
mean_age = df['Age'].mean()  # Mean
median_age = df['Age'].median()  # Median
std_age = df['Age'].std()  # Standard deviation
 min_age = df['Age'].min()  # Minimum value
 max_age = df['Age'].max()  # Maximum value

 print("\nStatistical operations on 'Age' column:")
 print(f"Mean: {mean_age}")
 print(f"Median: {median_age}")
```

```
print(f"Standard Deviation: {std_age}")
print(f"Min: {min_age}")
print(f"Max: {max_age}")
```

output:

```
First 5 records:
        Name  Age         City
0      Akash   25      Kurnool
1       Balu   30   Ananthapur
2    Charlie   35      Nandyal
3     Dally    40    Hyderabad
4  Evangilin   45      Chennai

Last 5 records:
        Name  Age         City
1       Balu   30   Ananthapur
2    Charlie   35      Nandyal
3     Dally    40    Hyderabad
4  Evangilin   45      Chennai
5   Florence   36    Bangalore

Shape of the DataFrame: (6, 3)

Index of the DataFrame: RangeIndex(start=0, stop=6, step=1)

Columns of the DataFrame: Index(['Name', 'Age', 'City'], dtype='object')

Rows where 'Age' > 30:
        Name  Age        City
2    Charlie   35     Nandyal
3     Dally    40   Hyderabad
4  Evangilin   45     Chennai
5   Florence   36   Bangalore

DataFrame after dropping 'City' column:
        Name  Age
0      Akash   25
1       Balu   30
2    Charlie   35
3     Dally    40
4  Evangilin   45
5   Florence   36

DataFrame sorted by 'Age' (ascending):
        Name  Age         City
0      Akash   25      Kurnool
1       Balu   30   Ananthapur
2    Charlie   35      Nandyal
5   Florence   36    Bangalore
3     Dally    40    Hyderabad
4  Evangilin   45      Chennai

DataFrame with 'Age Rank' column:
        Name  Age         City  Age Rank
0      Akash   25      Kurnool       1.0
1       Balu   30   Ananthapur       2.0
2    Charlie   35      Nandyal       3.0
3     Dally    40    Hyderabad       5.0
```

```
4  Evangilin   45      Chennai       6.0
5   Florence   36    Bangalore       4.0

Statistical operations on 'Age' column:
Mean: 35.166666666666664
Median: 35.5
Standard Deviation: 7.082843120291926
Min: 25
Max: 45
```

6. Import any CSV file to Pandas Data Frame and perform the following:

- Handle missing data by detecting and dropping/ filling missing values.

Emp.csv

Name,Age,City,Salary
Aarav,25,Mumbai,50000
Bhavna,30,New Delhi,55000
Chirag,35,Bangalore,60000
Deepak,40,Hyderabad,
Esha,,Kolkata,80000
Farhan,45,Mumbai,52000
Gauri,32,Bangalore,59000
Harsh,,New Delhi,65000
Ishita,50,Hyderabad,75000
Jayant,36,,72000

Python code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('d:/emp.csv')
print("Missing data in each column:")
print(df.isnull().sum())

df_dropped = df.dropna()
print("\nData after dropping rows with missing values:")
print(df_dropped.head())

df_filled = df.fillna(df.mean())
print("\nData after filling missing values with mean:")
```

print(df_filled)

output:

```
Missing data in each column:
Name      0
Age       2
City      1
Salary    1
dtype: int64

Data after dropping rows with missing values:
     Name   Age       City   Salary
0   Aarav  25.0     Mumbai  50000.0
1  Bhavna  30.0  New Delhi  55000.0
2  Chirag  35.0  Bangalore  60000.0
5  Farhan  45.0     Mumbai  52000.0
6   Gauri  32.0  Bangalore  59000.0

Data after filling missing values with mean:
     Name     Age       City        Salary
0   Aarav  25.000     Mumbai  50000.000000
1  Bhavna  30.000  New Delhi  55000.000000
2  Chirag  35.000  Bangalore  60000.000000
3  Deepak  40.000  Hyderabad  63111.111111
4    Esha  36.625    Kolkata  80000.000000
5  Farhan  45.000     Mumbai  52000.000000
6   Gauri  32.000  Bangalore  59000.000000
7   Harsh  36.625  New Delhi  65000.000000
8  Ishita  50.000  Hyderabad  75000.000000
9  Jayant  36.000        NaN  72000.000000
```

7. Import any CSV file to Pandas Data Frame and perform the following:
   a. Transform data using apply () and map() method.
   b. Perform Vectorized String operations on Pandas Series.

City.csv

```
Name,Age,City,Salary
Aarav,25,Mumbai,50000
Bhavna,30,New Delhi,55000
Chirag,35,Bangalore,60000
Deepak,40,Hyderabad,56000
Esha,30,Kolkata,80000
Farhan,45,Mumbai,52000
Gauri,32,Bangalore,59000
Harsh,29,New Delhi,65000
Ishita,50,Hyderabad,75000
Jayant,36,Kolkata,72000
```

Python code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
df = pd.read_csv('d:/city.csv')

df['Age Transformed'] = df['Age'].apply(lambda x: x * 2 if x < 30 else x)

df['City Abbr'] = df['City'].map({'Mumbai': 'MUM', 'New Delhi': 'ND',
'Bangalore': 'BAN','Hyderabad':'HYD','Kolkata':'KOL'})
print("\nData with transformations using apply() and map():")
print(df[['Name', 'Age', 'Age Transformed', 'City', 'City Abbr']].head())

df['City Uppercase'] = df['City'].str.upper()
print("\nData with vectorized string operations (city names in uppercase):")
print(df[['City', 'City Uppercase']].head())
```

output:

```
Data with transformations using apply() and map():
     Name  Age  Age Transformed       City City Abbr
0   Aarav   25               50     Mumbai       MUM
1  Bhavna   30               30  New Delhi        ND
2  Chirag   35               35  Bangalore       BAN
3  Deepak   40               40  Hyderabad       HYD
4    Esha   30               30    Kolkata       KOL

Data with vectorized string operations (city names in uppercase):
        City City Uppercase
0     Mumbai         MUMBAI
1  New Delhi      NEW DELHI
2  Bangalore      BANGALORE
3  Hyderabad      HYDERABAD
4    Kolkata        KOLKATA
```

8. Visualize data using Line Plots and Bar Plots.

```
import matplotlib.pyplot as plt

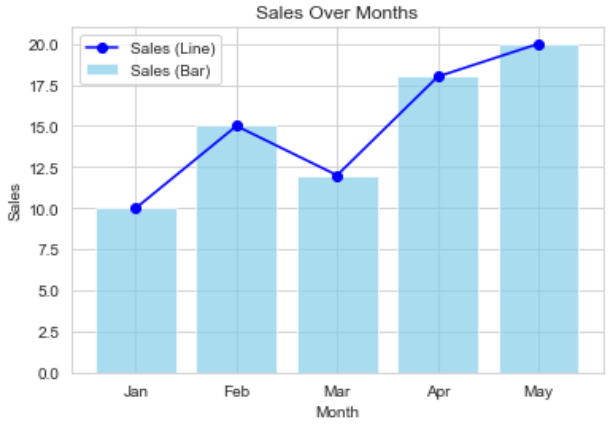
months = ["Jan", "Feb", "Mar", "Apr", "May"]
sales = [10, 15, 12, 18, 20]

plt.bar(months, sales, color="skyblue", alpha=0.7, label="Sales (Bar)")

plt.plot(months, sales, color="blue", marker="o", label="Sales (Line)")

plt.title("Sales Over Months")
```

```
plt.xlabel("Month")
plt.ylabel("Sales")
plt.legend()
plt.show()
```



9. **Visualize data using Histograms and Box Plot.**

```
import matplotlib.pyplot as plt
import numpy as np

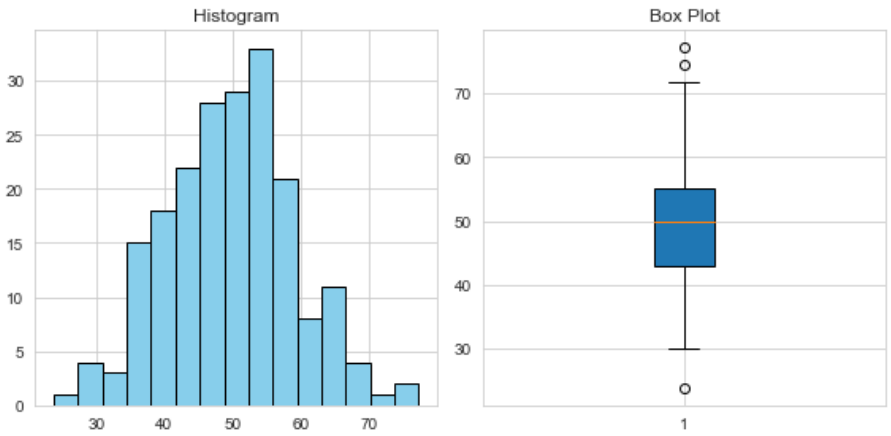
np.random.seed(42)
data = np.random.normal(50, 10, 200)

plt.figure(figsize=(8,4))

plt.subplot(1, 2, 1)
plt.hist(data, bins=15, color="skyblue", edgecolor="black")
plt.title("Histogram")

plt.subplot(1, 2, 2)
plt.boxplot(data, vert=True, patch_artist=True)
plt.title("Box Plot")

plt.tight_layout()
plt.show()
```



10. **Visualize data using Density Plots and Scatter Plots.**

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
x = np.random.normal(5, 2, 100)
y = 0.5 * x + np.random.normal(0, 1, 100)

plt.scatter(x, y, color="royalblue", alpha=0.6)
plt.title("Simple Scatter Plot")
plt.xlabel("X Value")
plt.ylabel("Y Value")
plt.show()

np.random.seed(42)
data = np.random.normal(50, 10, 200)

sns.kdeplot(x=data, fill=True, color="teal", alpha=0.5)
plt.title("Density Plot")
plt.show()
```

